Ejercicios IV

DISEÑO DE SOFTWARE

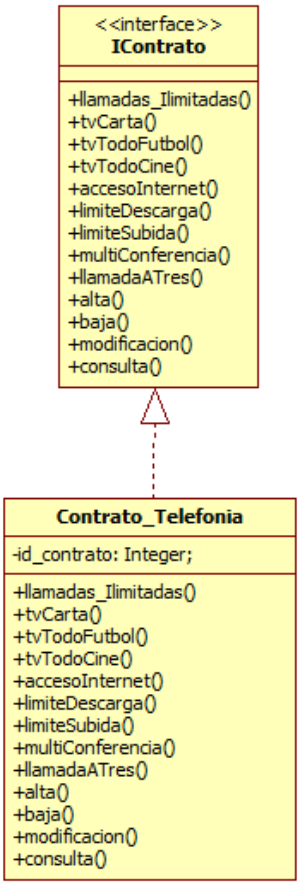
Fundamentos de Ingeniería del Software

Nota:

Aunque en un diagrama de clases de diseño completo deben aparecer todos los atributos y métodos, así como el tipo de cada atributo y todos los parámetros de todos los métodos, los diagramas de clases de diseño de los ejercicios incluidos en este documento no siempre incluyen toda esta información a fin de simplificar y facilitar la comprensión de los enunciados y soluciones de los ejercicios. No obstante, cabe destacar que estos diagramas siempre incluyen toda la información necesaria para comprender el enunciado de los ejercicios y sus soluciones.

Ejercicio 1

La compañía de telecomunicaciones VozUPM quiere desarrollar una aplicación para gestionar sus contratos. Esta compañía ofrece servicios de telefonía, Internet y televisión. Por lo tanto, el contrato de cada cliente debe especificar la combinación de los servicios de telefonía, Internet y televisión que éste tiene contratados. Tras finalizar la fase de análisis del desarrollo de la aplicación de gestión de contratos, se ha elaborado, ya en la fase de diseño, el siguiente diagrama de clases de diseño:



1. Indique de manera razonada si el diseño proporcionado en el enunciado cumple el principio de responsabilidad única.
2. Aplique el principio de segregación de interfaces sobre el diagrama de clases de diseño proporcionado en el enunciado e indique de manera justificada si la nueva solución mejora la cohesión.
3. Mejore el diagrama de clases de diseño solicitado en el apartado 2 aplicando los principios de diseño que estime oportunos e indique de manera justificada si la solución propuesta mejora la cohesión.
4. VozUPM ha visto que se ha generado una demanda de nuevos servicios de telefonía y televisión. En particular, los usuarios han demandado la incorporación de servicios de identificación de llamadas y de bloqueo

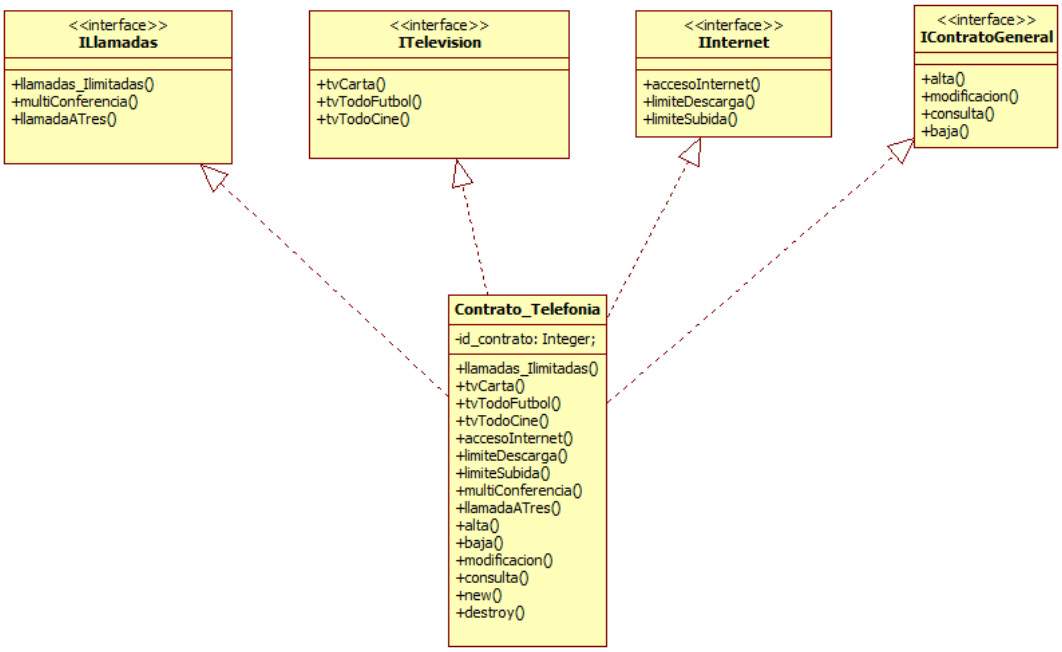
de llamadas. En el caso del servicio de televisión, se necesita la adopción de un control parental para evitar que los menores tengan acceso a determinados contenidos. Modifique el diagrama de clases de diseño solicitado en el apartado 3 a fin de añadir a la aplicación de gestión de contratos los nuevos servicios mencionados respetando el principio Abierto/Cerrado.

Solución

Apartado 1.

En el diseño propuesto, la clase "Contrato_Telefonia" tiene cuatro responsabilidades principales: gestión (alta, baja, modificación y consulta) de contratos, servicio de telefonía, servicio de Internet y servicio de televisión. Por lo tanto, como esta clase tiene claramente más de una responsabilidad, el diseño no cumple el principio de responsabilidad única.

Apartado 2.



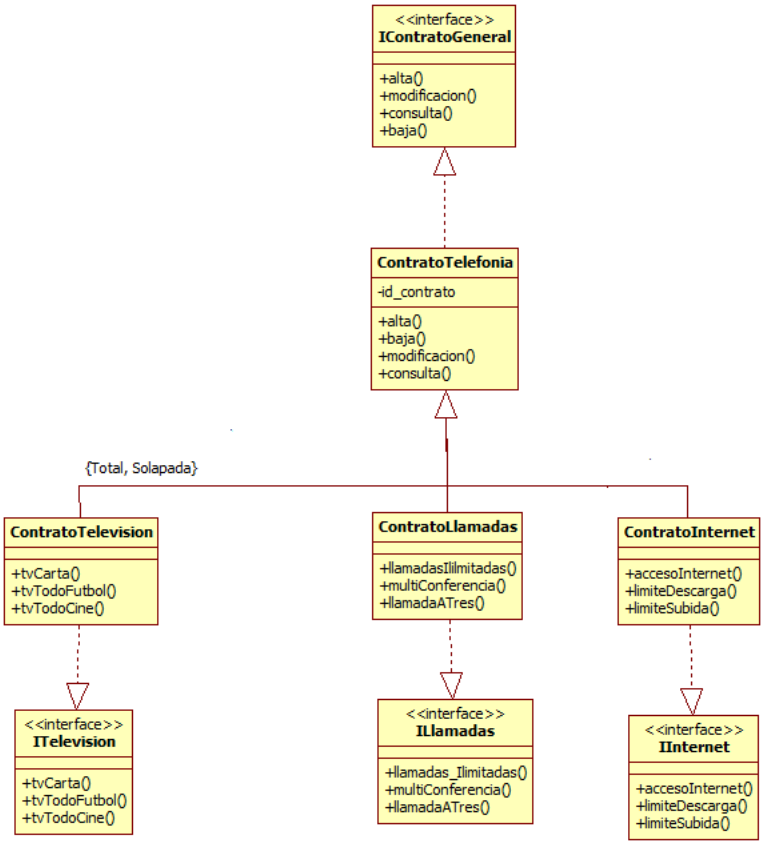
En el diseño propuesto en el enunciado, la clase "Contrato_Telefonia" implementa la interfaz "IContrato", que es una interfaz pesada ya que contiene los métodos relativos a las cuatro responsabilidades identificadas en el apartado anterior (gestión de contratos, servicio de telefonía, servicio de Internet y servicio de televisión). En este caso, la aplicación del principio de segregación de interfaces consiste en separar la interfaz pesada "IContrato" en cuatro interfaces más ligeras, cada una de las cuales contiene los métodos relativos a una de las responsabilidades identificadas.

La solución propuesta en este apartado no varía la cohesión respecto al diseño inicial ya que el cuerpo de la clase “Contrato_Telefonia” sigue siendo el mismo. Los métodos de esta clase no han variado por lo que siguen estando igual de relacionados que antes.

Apartado 3.

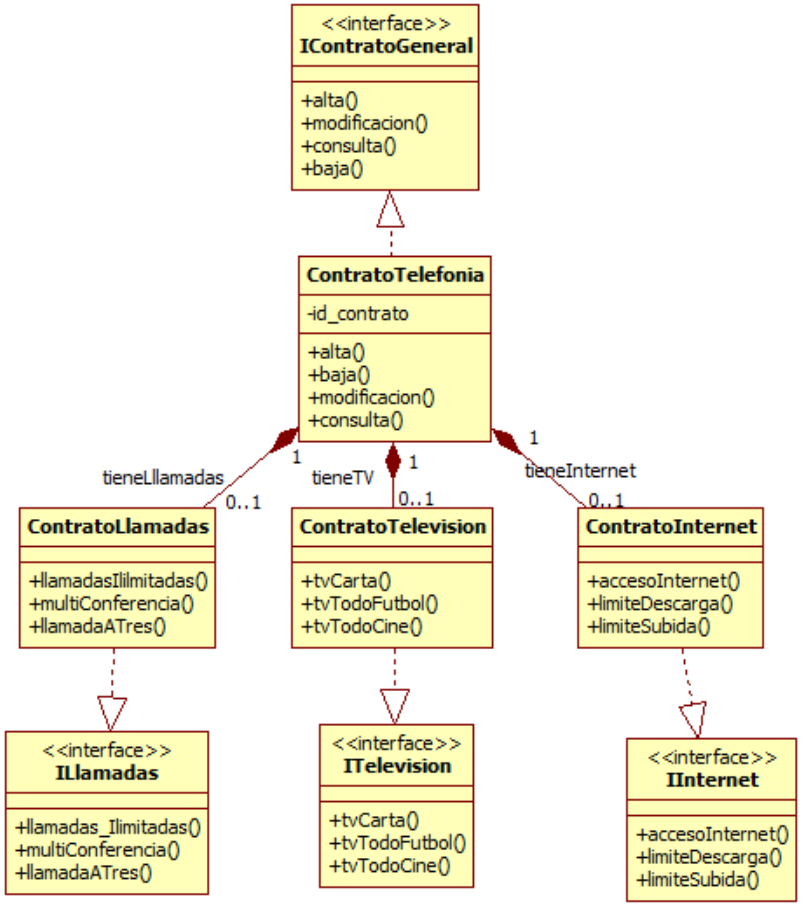
El diseño propuesto se puede mejorar aplicando el principio de responsabilidad única y el principio de abstracción, el cual puede ser aplicado mediante especialización o descomposición de la clase “Contrato_Telefonia”.

La solución aplicando el principio de abstracción por especialización sería la siguiente:



Esta solución tiene como inconveniente que dificulta los cambios de contrato, ya que un cambio de contrato implicaría destruir un objeto y crear otro transfiriendo el estado.

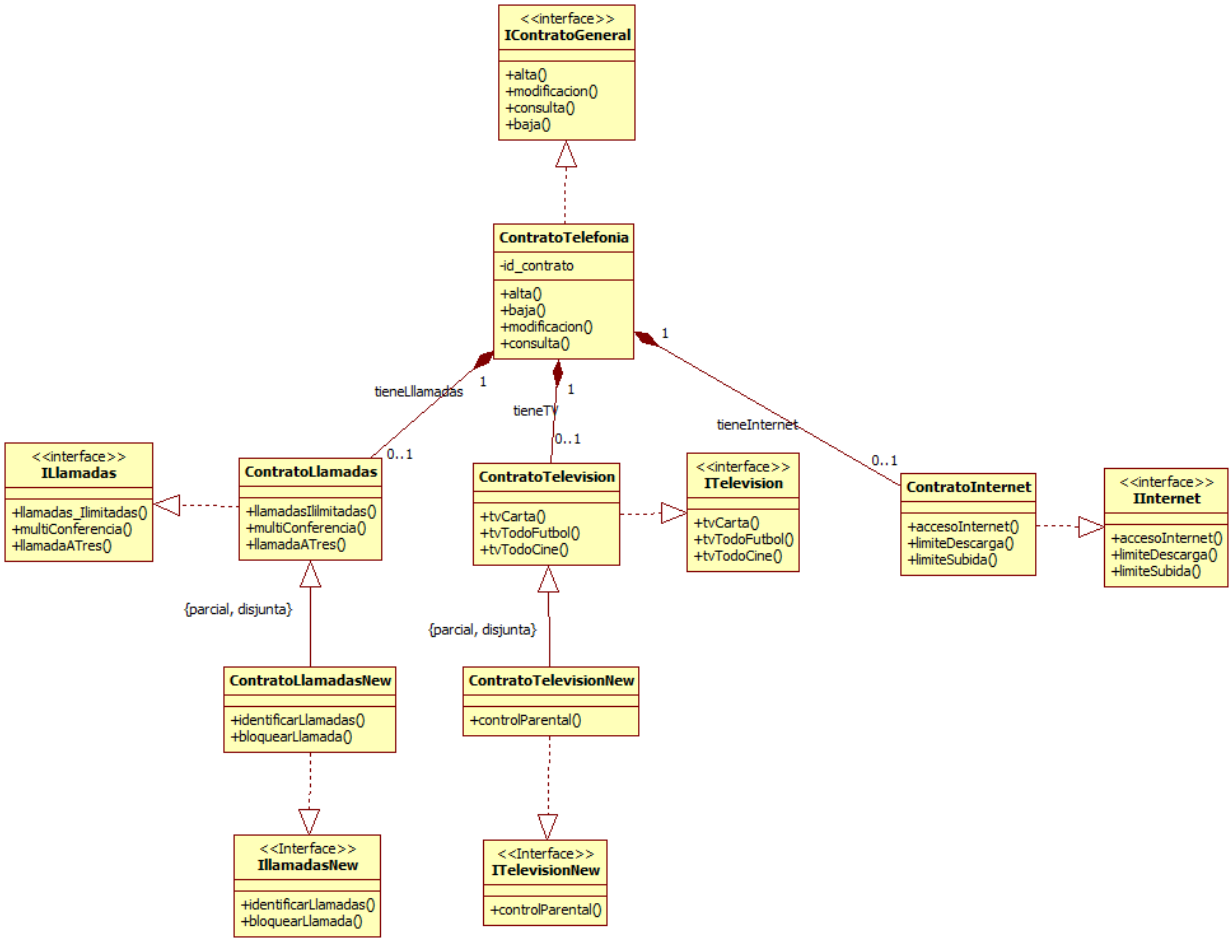
La solución aplicando el principio de abstracción por descomposición sería la siguiente:



Esta solución tiene como inconveniente que la clase "ContratoTelefonia" (que es la más abstracta) depende de las clases más concretas ("ContratoLlamadas", "ContratoTelevision" y "Contrato Internet"), por lo que tiene un alto acoplamiento. No obstante, los cambios de contrato serían más sencillos que con la solución alternativa.

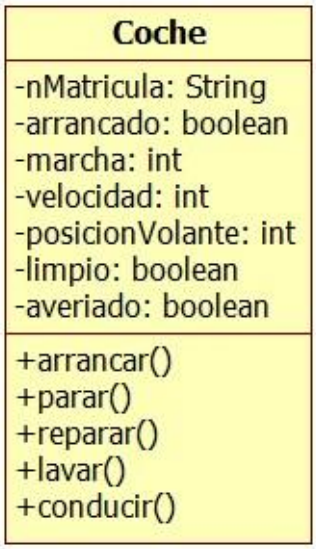
En este caso, las dos soluciones propuestas mejoran (es decir, aumentan) la cohesión respecto al diseño inicial porque los métodos que contenía la clase "Contrato_Telefonia" han sido repartidos en diferentes clases, provocando que ahora los métodos de cada clase estén más relacionados entre sí de lo que lo estaban los métodos de la clase "Contrato_Telefonia" en el diseño inicial.

Apartado 4.


<<interface>>
IContratoGeneral
+alta()
+modificacion()
+consulta()
+baja()
ContratoTelefonia
-id_contrato
+alta()
+baja()
+modificacion()
+consulta()
1
tieneLlamadas
1
tieneTV
1
tieneInternet
0..1
0..1
0..1
<<interface>>
ILlamadas
+llamadas_Ilimitadas()
+multiConferencia()
+llamadaATres()
ContratoLlamadas
+llamadasIlilmitadas()
+multiConferencia()
+llamadaATres()
ContratoTelevision
+tvCarta()
+tvTodoFutbol()
+tvTodoCine()
<<interface>>
ITelevision
+tvCarta()
+tvTodoFutbol()
+tvTodoCine()
ContratoInternet
+accesoInternet()
+limiteDescarga()
+limiteSubida()
<<interface>>
IInternet
+accesoInternet()
+limiteDescarga()
+limiteSubida()
{parcial, disjunta}
{parcial, disjunta}
ContratoLlamadasNew
+identificarLlamadas()
+bloquearLlamada()
ContratoTelevisionNew
+controlParental()
<<Interface>>
IllamadasNew
+identificarLlamadas()
+bloquearLlamada()
<<Interface>>
ITelevisionNew
+controlParental()

Ejercicio 2

A continuación, se muestra un fragmento de un diagrama de clases de diseño que ha realizado un equipo de desarrollo de software a fin de modelar un sistema software relacionado con el mundo del automóvil:



1. Explique por qué el diseño proporcionado en el enunciado no cumple el principio de responsabilidad única.
2. Aplique el principio de responsabilidad única sobre el diagrama de clases de diseño proporcionado en el enunciado.

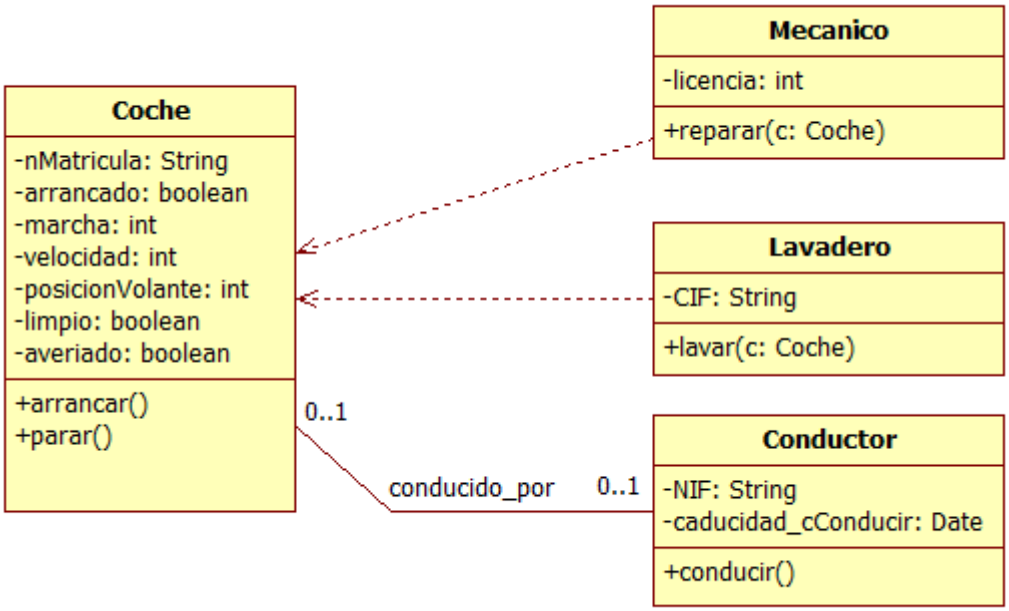
Solución

Apartado 1.

En el diseño propuesto, la clase "Coche" tiene múltiples responsabilidades. Algunas de ellas son claramente propias de un coche (arrancar() y parar()), mientras que otras (reparar(), lavar() y conducir()) no parecen ser responsabilidades propias de un coche convencional. El diseño propuesto estaría asumiendo que todo coche es capaz de repararse y lavarse a sí mismo, así como que todos los coches son vehículos autónomos capaces de circular sin conductor. En definitiva, la clase "Coche" está asumiendo varias responsabilidades que no le corresponden, lo cual ocasiona que se viole el principio de responsabilidad única y, además, que esta clase tenga una baja cohesión.

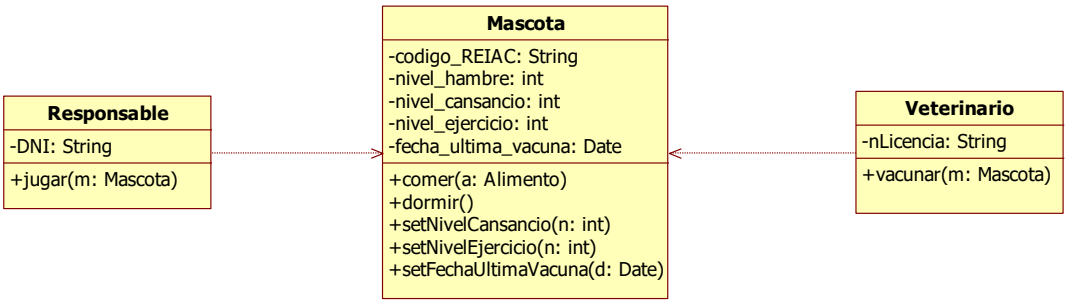
Apartado 2.

Para que el diseño cumpla con el principio de responsabilidad única, es necesario trasladar a nuevas clases aquellas responsabilidades de la clase "Coche" que no sean propias de los coches, es decir, las responsabilidades de reparar, lavar y conducir. En este caso se propone la siguiente solución:



Ejercicio 3

Un equipo de ingenieros de software está desarrollando un sistema software para una clínica veterinaria. Este sistema debe almacenar la información de todos los veterinarios que trabajan en la clínica, las mascotas a las que éstos atienden y los responsables legales de dichas mascotas. El responsable legal de una mascota puede jugar con ella pero no puede vacunarla, mientras que los veterinarios pueden vacunar a las mascotas pero nunca juegan con ellas. A continuación, se muestra un fragmento del diagrama de clases de diseño que ha realizado el equipo de ingenieros de software para modelar el sistema para la clínica veterinaria.



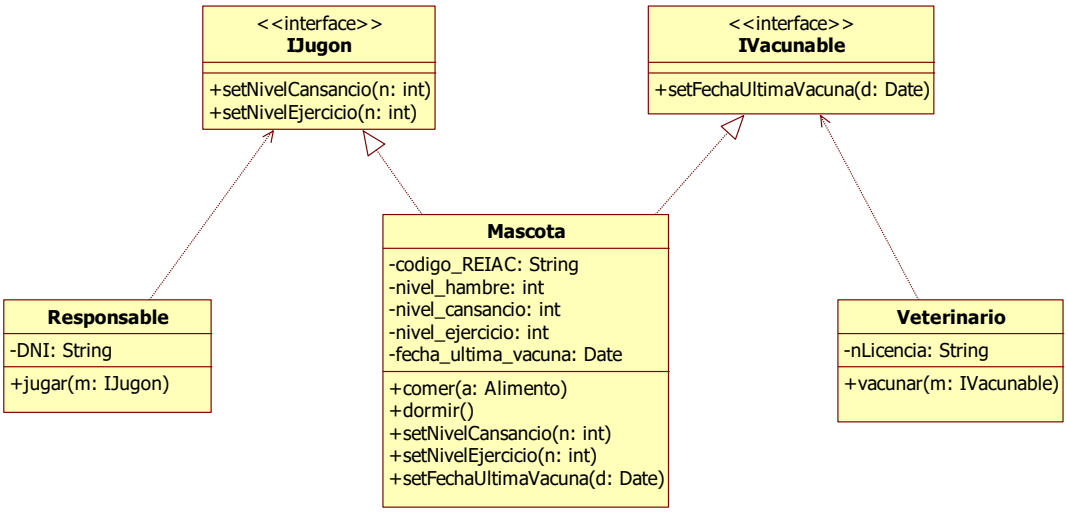
Cuando se ejecuta el método "comer(a: Alimento)" de un objeto de la clase "Mascota" disminuye su nivel de hambre, mientras que cuando se ejecuta el método "dormir" disminuye su nivel de cansancio. Cuando se ejecuta el método "jugar (m: Mascota)" de la clase "Responsable" se actualiza el nivel de ejercicio y cansancio del objeto de la clase "Mascota" pasado como parámetro (es decir, del objeto m). Por último, cuando se ejecuta el método "vacunar (m: Mascota)" de la clase "Veterinario" se actualiza la fecha de la última vacuna del objeto m.

1. Aplique los principios de inversión de dependencias y segregación de interfaces sobre el diagrama de clases de diseño proporcionado en el enunciado.
2. Aplique nuevamente los principios de inversión de dependencias y segregación de interfaces sobre el diagrama de clases de diseño proporcionado en el enunciado asumiendo que:
    a. Existe un método "alimentar(m: Mascota, a: Alimento)" tanto en la clase "Responsable" como en la clase "Veterinario".
    b. Cuando se ejecuta este método se origina una llamada al método "comer(a: Alimento)" de la clase "Mascota".

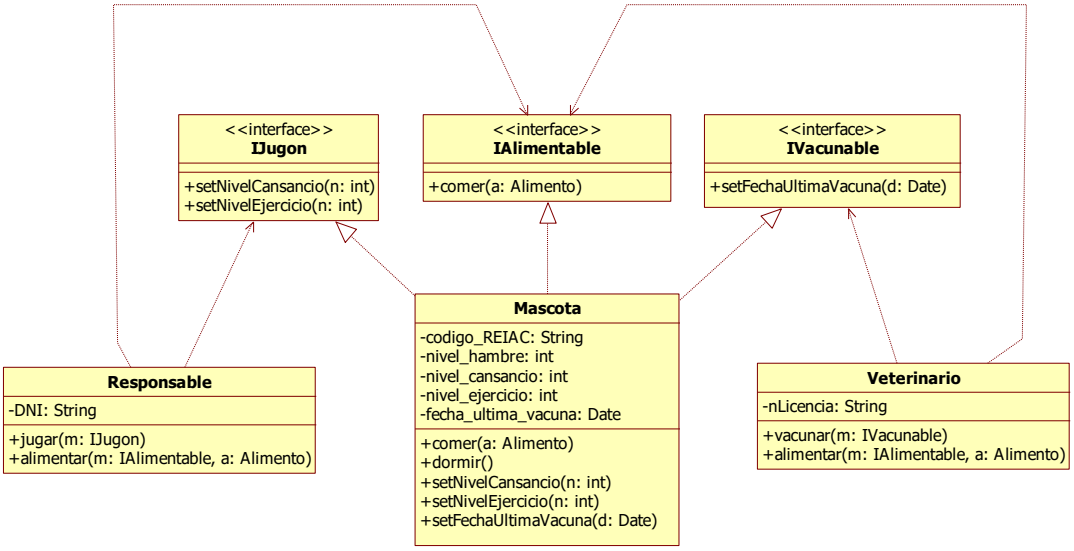
3. Aplique nuevamente los principios de inversión de dependencias y segregación de interfaces sobre el diagrama de clases de diseño proporcionado en el enunciado asumiendo que:
    a. Cuando se ejecuta el método "vacunar (m: Mascota)" de la clase "Veterinario", no solo se actualiza la fecha de la última vacuna del objeto m, sino que también se modifica su nivel de cansancio.
4. Aplique nuevamente los principios de inversión de dependencias y segregación de interfaces sobre el diagrama de clases de diseño proporcionado en el enunciado asumiendo que:
    a. El sistema software necesita conocer, para cada persona legalmente responsable de una o más mascotas, cuales son estas mascotas.
    b. El sistema no necesita conocer, dada una mascota, cuál es su responsable legal.

Solución

Apartado 1.

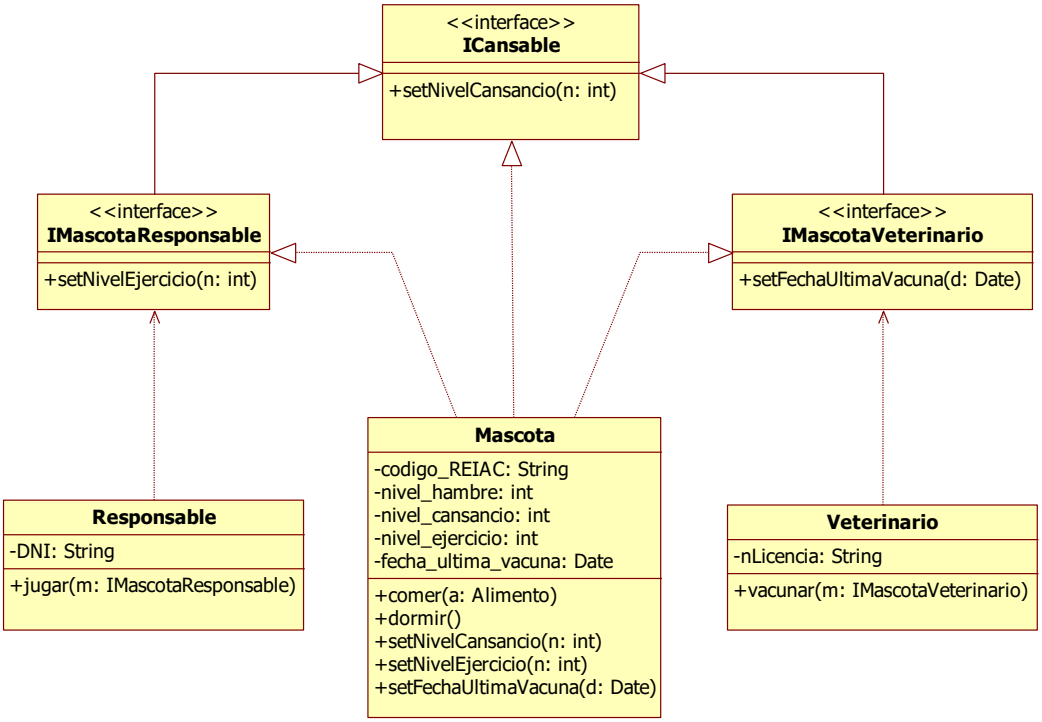


Apartado 2.



Apartado 3.

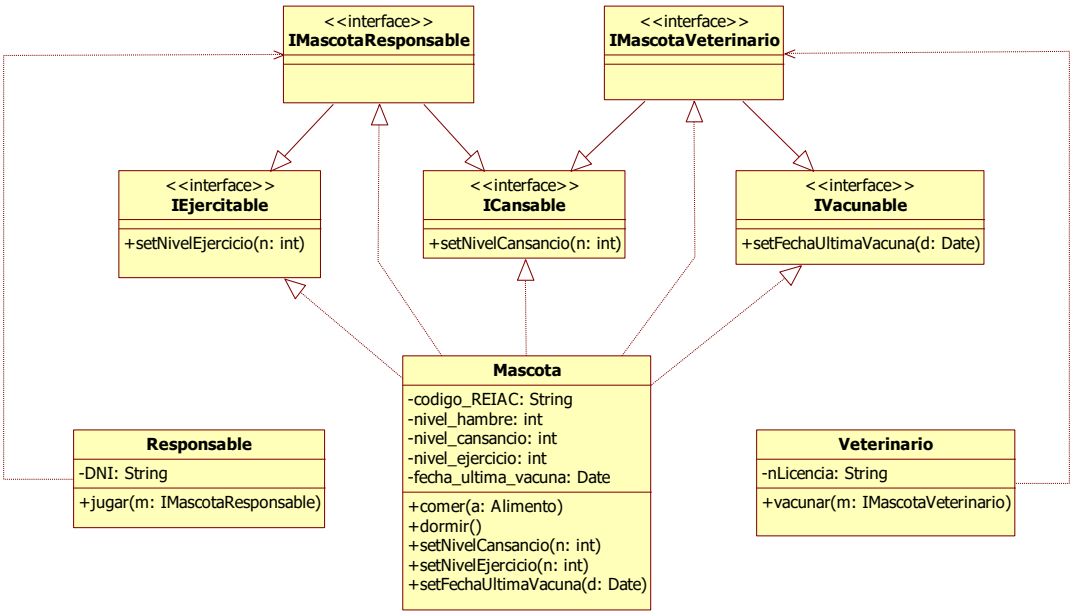
Opción a (preferible): empleando herencia de interfaces.



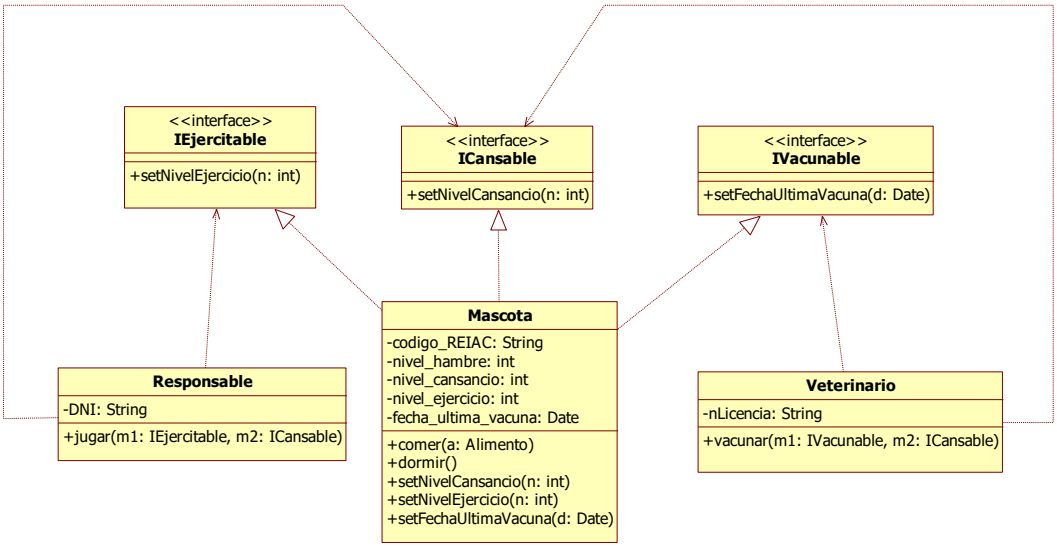
Esta solución cumple el principio de segregación de interfaces puesto que se cumplen las siguientes dos condiciones:

- Ninguna clase se ve obligada a implementar métodos que no necesita realmente.
- Las clases "Responsable" y "Veterinario" solo dependen de los métodos de "Mascota" que realmente usan.

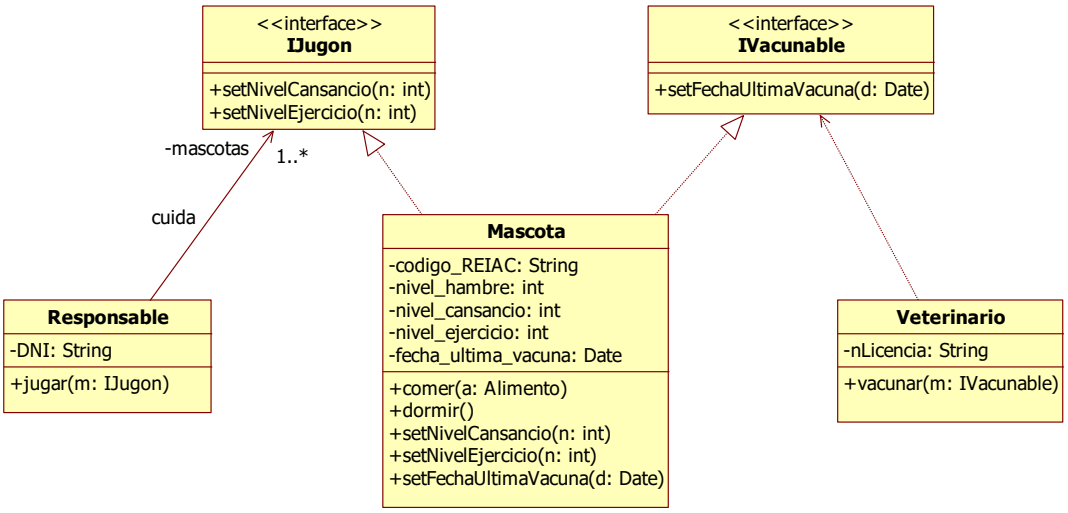
Una solución alternativa empleando herencia de interfaces sería la siguiente:



Opción b: empleando atributos adicionales en los métodos.

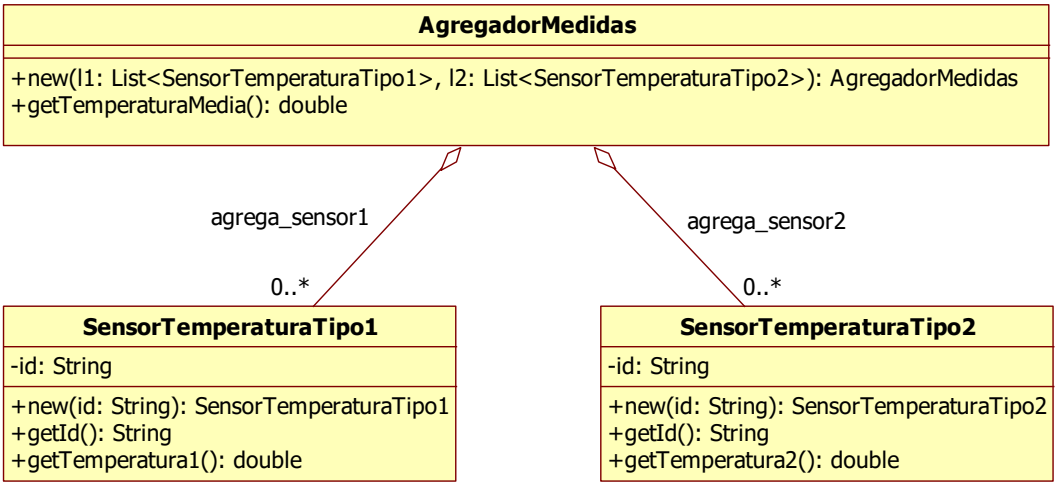


Apartado 4.


<<interface>>
IJugon
+setNivelCansancio(n: int)
+setNivelEjercicio(n: int)
<<interface>>
IVacunable
+setFechaUltimaVacuna(d: Date)
-mascotas
1..*
cuida
Mascota
-codigo_REIAC: String
-nivel_hambre: int
-nivel_cansancio: int
-nivel_ejercicio: int
-fecha_ultima_vacuna: Date
+comer(a: Alimento)
+dormir()
+setNivelCansancio(n: int)
+setNivelEjercicio(n: int)
+setFechaUltimaVacuna(d: Date)
Responsable
-DNI: String
+jugar(m: IJugon)
Veterinario
-nLicencia: String
+vacunar(m: IVacunable)

Ejercicio 4

Se ha diseñado e implementado un sistema para la obtención y agregación de medidas obtenidas por sensores de temperatura de diferentes tipos. Inicialmente, se ha considerado que el sistema debe soportar únicamente dos tipos de sensores de temperatura (tipo 1 y tipo 2). Cada uno de estos tipos de sensores, al ser de un fabricante diferente, obtiene las medidas de temperatura de forma distinta. Se ha propuesto el siguiente diagrama de clases de diseño para modelar el sistema descrito:



La implementación de los métodos getTemperatura1() y getTemperatura2() es diferente porque cada tipo de sensor obtiene las medidas de temperatura de forma distinta. La implementación de la clase AgregadorMedidas es la siguiente:

```
import java.util.List;

public class AgregadorMedidas {
  private List<SensorTemperaturaTipo1> l1;
  private List<SensorTemperaturaTipo2> l2;

  public AgregadorMedidas(List<SensorTemperaturaTipo1> l1,
                          List<SensorTemperaturaTipo2> l2) {
    this.l1 = l1;
    this.l2 = l2;
  }

  public double getTemperaturaMedia() {
    double temperatura = 0;
    int nSensores = 0;
    for(int i = 0; i<l1.size(); i++) {
      nSensores++;
      temperatura += l1.get(i).getTemperatura1();
    }
    for(int j = 0; j<l2.size(); j++) {
      nSensores++;
      temperatura += l2.get(j).getTemperatura2();
    }
    return temperatura/nSensores;
  }
}
```

1. Indique los cambios que, de acuerdo al diseño propuesto, habría que realizar en el sistema para que éste pudiera soportar un tercer tipo de sensor de temperatura.
2. Indique si el diseño del sistema es adecuado.
3. Mejore el diagrama de clases de diseño proporcionado en el enunciado aplicando los principios de diseño que estime oportunos.
4. Escriba la implementación de la clase "AgregadorMedidas" en caso de que el sistema se hubiera implementado de acuerdo al diseño solicitado en el apartado 3.

Solución

Apartado 1.

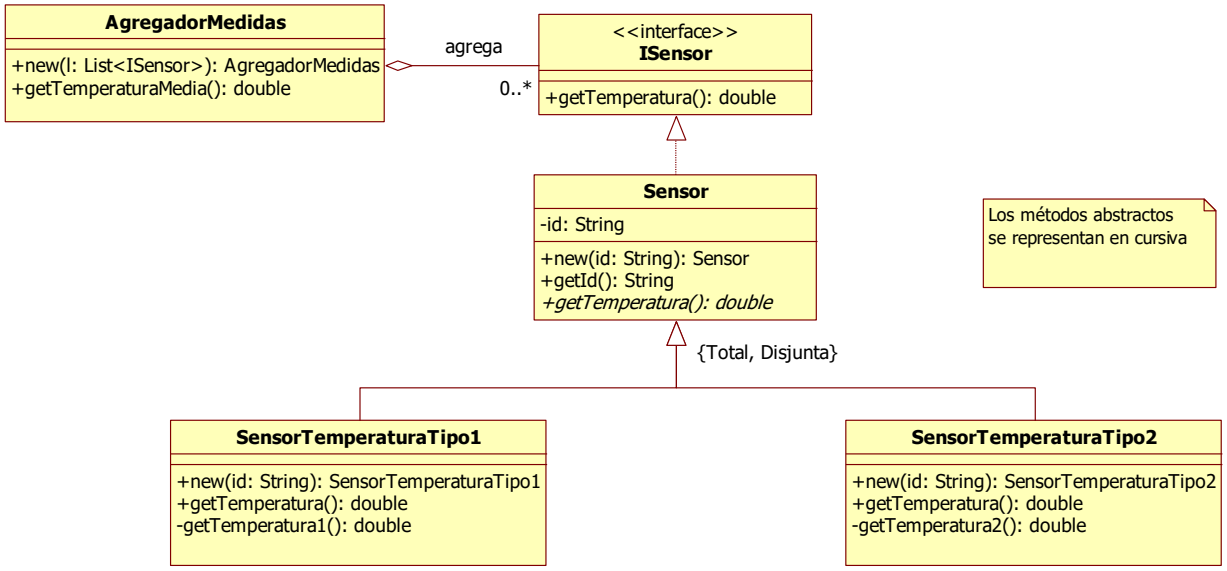
De acuerdo al diseño propuesto, para que el sistema pudiera soportar un tercer tipo de sensor de temperatura habría que:

- Crear una nueva clase "SensorTemperaturaTipo3" que implementase un método "getTemperatura3()".
- Modificar la clase "AgregadorMedidas": sería necesario añadir un atributo "List<SensorTemperaturaTipo3> l3", modificar el constructor y también modificar el método "getTemperaturaMedia()".

Apartado 2.

La clase "AgregadorMedidas" depende directamente de la implementación de las clases que representan a los sensores (p. ej., de "SensorTemperaturaTipo1"). Además, la adición de soporte para un nuevo sensor implica modificar la clase "AgregadorMedidas". En definitiva, la clase "AgregadorMedidas" está fuertemente acoplada a las clases de los sensores. Como este acoplamiento es altamente dañino y puede ser fácilmente evitado, podemos afirmar que el diseño del sistema no es adecuado.

Apartado 3.



El diagrama de clases de diseño se ha mejorado aplicando tres principios de diseño:

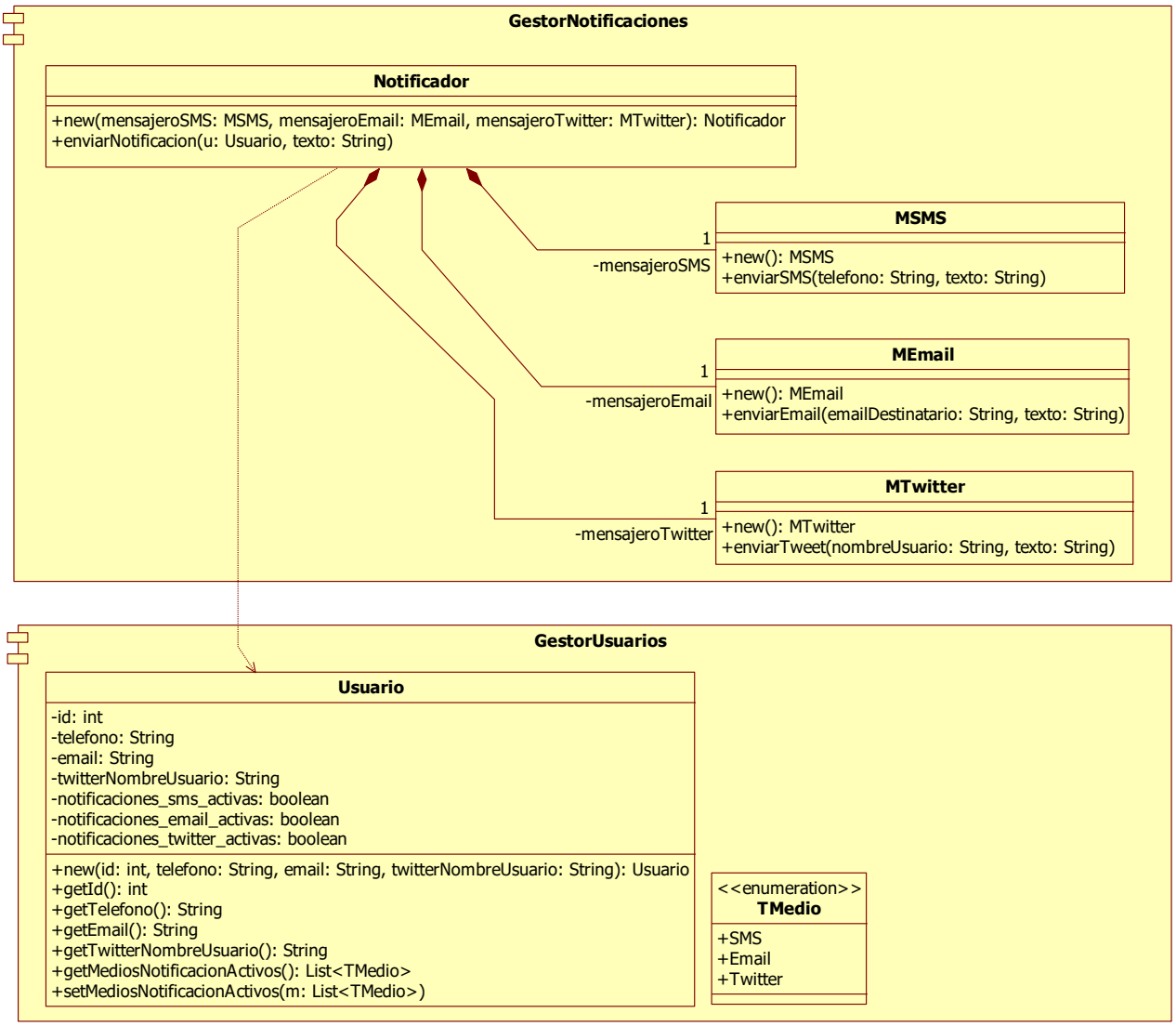
- **Abstracción por generalización.** Se ha creado la clase "Sensor" y se han incluido en ella todos los atributos y métodos comunes a todos los tipos de sensor.
- **Sustitución de Liskov.** Se ha añadido un método abstracto getTemperatura() a la clase "Sensor" y, a cada una de sus clases hijas, un método concreto getTemperatura(). Cuando se llame al método getTemperatura() de "SensorTemperaturaTipo1" esto provocará una llamada al método getTemperatura1(). Igualmente, cuando se llame al método getTemperatura() de "SensorTemperaturaTipo2" esto provocará una llamada al método getTemperatura2(). Los métodos getTemperatura1() y getTemperatura2() pueden ser privados ahora, ya que solo son empleados por las clases que los definen. En este caso, el beneficio de aplicar el principio de sustitución de Liskov es que la clase "AgregadorMedidas" no tiene que conocer los diferentes tipos de sensores existentes y por tanto tampoco tiene que conocer detalles de las clases hijas de "Sensor".
- **Inversión de dependencias.** Se ha creado la interfaz "ISensor", la cual es implementada por la clase "Sensor". Posteriormente, se han eliminado las relaciones de agregación entre la clase "AgregadorMedidas" y las clases de los sensores y se han sustituido por una relación de agregación entre la clase "AgregadorMedidas" y la interfaz "ISensor", de tal forma que ahora la clase "AgregadorMedidas" ya no depende de la implementación de las clases de los sensores, sino que depende de la interfaz que éstas implementan.

Apartado 4.

```
import java.util.List;

public class AgregadorMedidas {
  private List<ISensor> l;

  public AgregadorMedidas(List<ISensor> l) {
    this.l = l;
  }

  public double getTemperaturaMedia() {
    double temperatura = 0;
    int nSensores = l.size();
    for(int i = 0; i<nSensores; i++) {
      temperatura += l.get(i).getTemperatura();
    }
    return temperatura/nSensores;
  }
}
```

Ejercicio 5

Se quiere desarrollar un sistema de notificaciones que sea capaz de enviar a un usuario una misma notificación a través de diferentes medios (SMS, email y Twitter). Este sistema solo deberá enviar notificaciones a los usuarios a través de aquellos medios que éstos tengan activos. Esto permitiría, por ejemplo, que si un usuario no desea recibir notificaciones por Twitter pero sí por email y SMS, éstas le sean enviadas únicamente mediante estos dos medios. Se ha propuesto el siguiente diagrama de clases de diseño (el cual indica el componente al que pertenece cada clase) para modelar el sistema descrito:

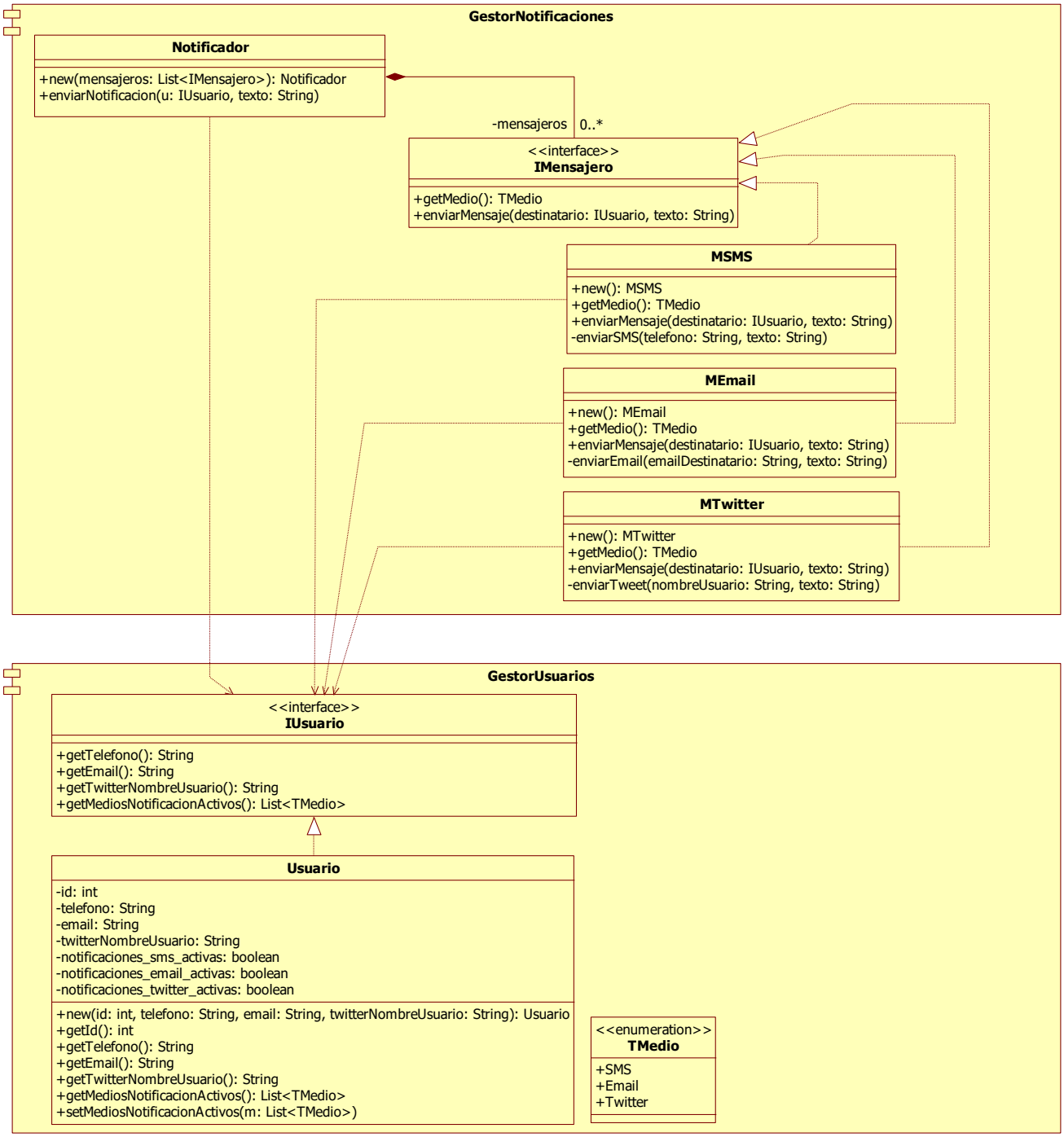


1. Aplique el principio de inversión de dependencias (y únicamente este principio) sobre el diagrama de clases de diseño proporcionado en el enunciado a fin de reducir lo máximo posible el acoplamiento (tanto entre componentes como dentro de cada componente). Indique de manera razonada si la nueva solución mejora el acoplamiento.

2. Realice el diagrama de componentes del sistema en base exclusivamente al diagrama de clases de diseño solicitado como solución del apartado 1.

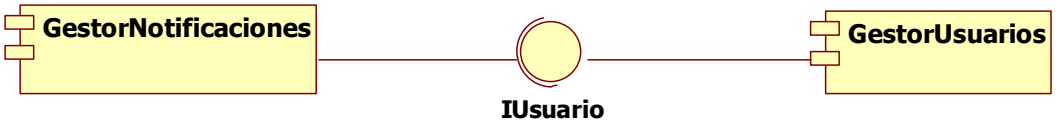
Solución

Apartado 1.



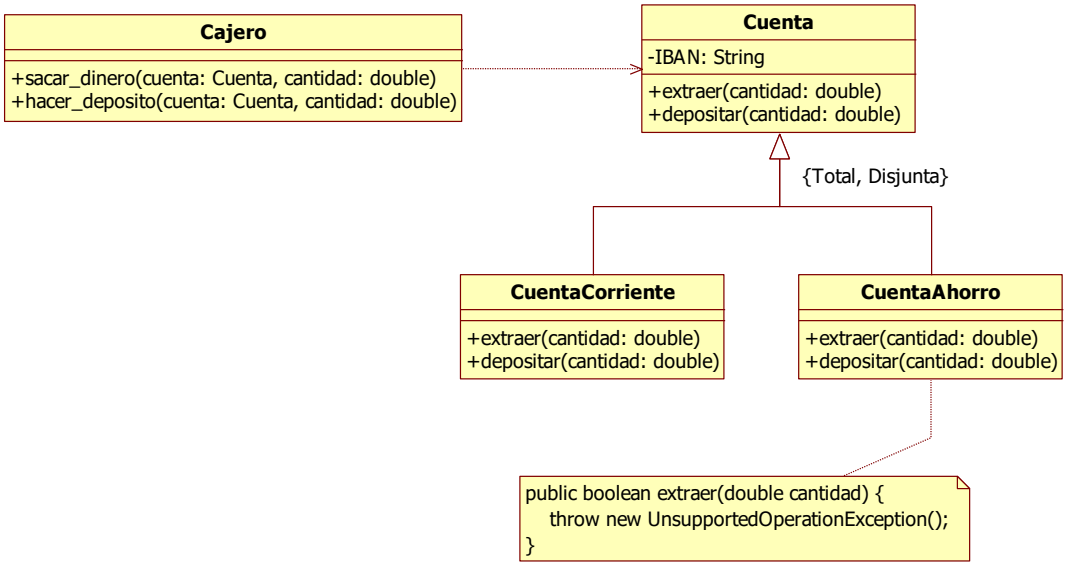
En la solución propuesta en el enunciado, el componente "GestorNotificaciones" depende de la implementación interna del componente "GestorUsuarios", mientras que en la solución propuesta en este apartado el componente "GestorNotificaciones" depende de una interfaz provista por el componente "GestorUsuarios". De manera similar, en la solución propuesta en el enunciado la clase "Notificador" está altamente acoplada a las clases de los medios de mensajería porque depende de su implementación, mientras que en la nueva solución la clase "Notificador" depende únicamente de una interfaz provista por dichas clases. Por tanto, la solución propuesta en este apartado mejora el acoplamiento del sistema porque reduce tanto el acoplamiento entre los componentes como el acoplamiento dentro del componente "GestorNotificaciones". El acoplamiento dentro del componente "GestorUsuarios" no puede reducirse porque este componente solo tiene una clase.

Apartado 2.



Ejercicio 6

Un determinado banco ofrece a sus clientes dos tipos de cuentas bancarias: cuenta corriente y cuenta de ahorro. Ambas cuentas permiten realizar ingresos en ellas, pero debido a la naturaleza ahorrativa de la cuenta de ahorros, no se permite sacar dinero de ella. El equipo técnico ha dado con el siguiente diagrama de clases de diseño para el sistema de gestión de cuentas del banco:



1. Explique por qué el diseño proporcionado en el enunciado no cumple el principio de sustitución de Liskov.
2. Aplique los cambios necesarios al diseño proporcionado en el enunciado para que éste cumpla con el principio de sustitución de Liskov.

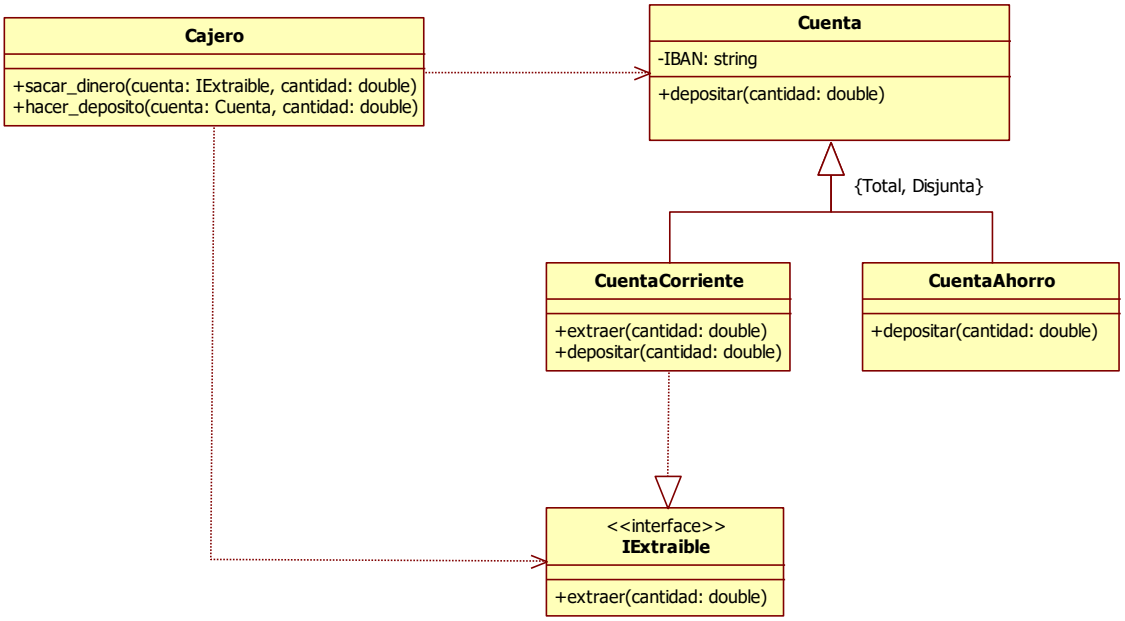
Solución

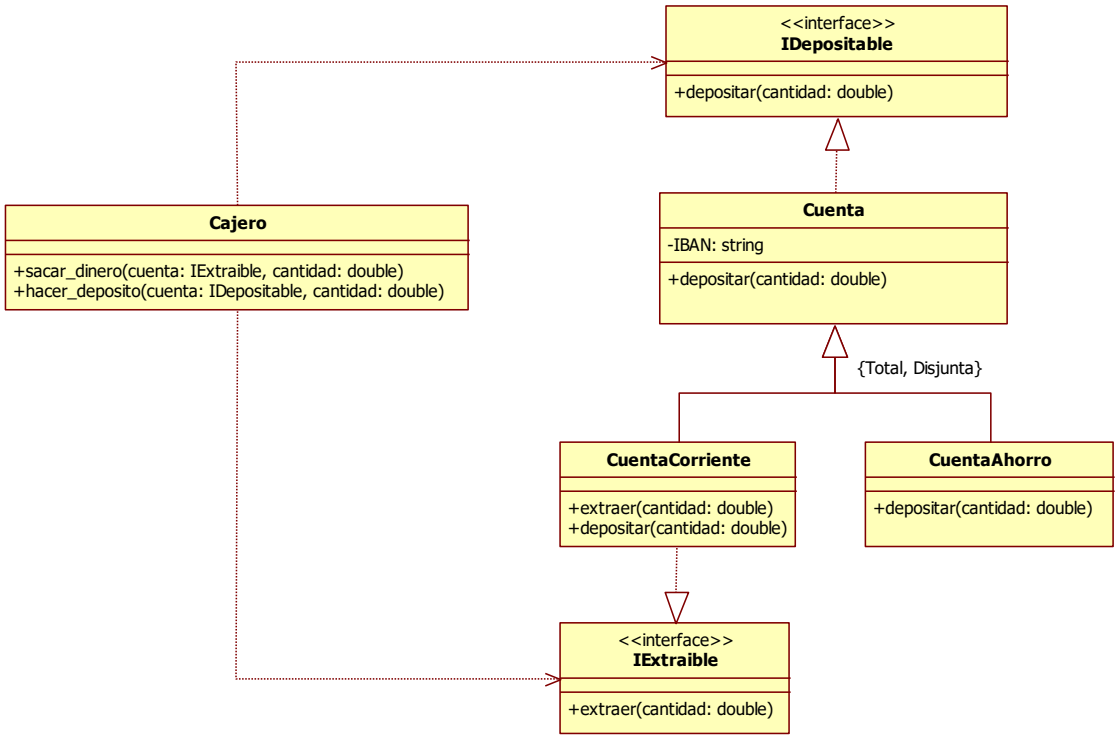
Apartado 1.

El principio de sustitución de Liskov nos obliga a asegurarnos de que cuando extendemos una clase no estamos alterando el comportamiento de la clase padre de la que deriva. El método "sacar_dinero" de la clase "Cajero" lanzará una excepción cuando el objeto pasado como parámetro sea de la clase "CuentaAhorro", lo cual es un comportamiento inesperado de una cuenta bancaria.

Apartado 2.

Una posible solución para que se cumpla el principio de sustitución de Liskov es la siguiente:



De esta forma especificamos que el comportamiento común de las cuentas del banco es que tienen un IBAN y que se puede depositar dinero en ellas, mientras que restringimos las cuentas de las que se puede sacar dinero a aquellas que cuenten con ese comportamiento mediante la interfaz IExtraible.

Otra posible solución que, además de cumplir el principio de sustitución de Liskov, reduce el acoplamiento, es la siguiente: